Probability, Log Loss, and Common RL Terms

Mathematical Foundations for Reinforcement Learning

1 Goal of This Note

Reinforcement learning uses probability everywhere. An agent may choose actions randomly, the environment may respond randomly, and the learning objective is usually an expectation.

The goal of this note is to make the common probabilistic language of RL feel concrete:

$$\boxed{\text{probabilities} \to \text{log-probabilities} \to \text{losses} \to \text{RL objectives}}$$

We will explain the terms that appear constantly in policy gradient, Q-learning, and modern RL for language models.

2 Random Variables

A random variable is a quantity whose value is uncertain. In reinforcement learning, the action can be a random variable:

$$A_t \in \{0, 1\}$$

For CartPole, we may use:

$$0 = \text{push left}, \qquad 1 = \text{push right}.$$

If the policy says:

$$\pi(a = 0 \mid s) = 0.30, \qquad \pi(a = 1 \mid s) = 0.70,$$

then the action is sampled from a distribution:

$$A_t \sim \pi(\cdot \mid s_t).$$

The dot means "all possible actions." So $\pi(\cdot \mid s_t)$ is the whole probability distribution over actions at state s_t.

3 Probability Distribution

A probability distribution assigns probabilities to possible outcomes. For a discrete action space:

$$\sum_a \pi(a \mid s) = 1, \qquad \pi(a \mid s) \geq 0.$$

For CartPole:

$$\pi(\cdot \mid s) = \begin{bmatrix} \pi(0 \mid s) \\ \pi(1 \mid s) \end{bmatrix} = \begin{bmatrix} 0.30 \\ 0.70 \end{bmatrix}.$$

The policy is not just one action. The policy is the full distribution.

4 Logits, Softmax, and Probabilities

Neural networks usually output logits, not probabilities. For CartPole, the neural network may output:

$$z = \begin{bmatrix} z_{\text{left}} \\ z_{\text{right}} \end{bmatrix} = \begin{bmatrix} 0.2 \\ 1.1 \end{bmatrix}.$$

These are raw scores. Softmax converts them into probabilities:

$$\pi(a = i \mid s) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

The largest logit gets the largest probability, but every action can still have nonzero probability.

```
import torch
from torch.distributions import Categorical

logits = torch.tensor([0.2, 1.1])
dist = Categorical(logits=logits)

print(dist.probs) # tensor([0.2891, 0.7109])
action = dist.sample()
log_prob = dist.log_prob(action)
```

5 Why Log-Probabilities Appear

In probability, products appear constantly. The probability of a trajectory is a product of many probabilities:

$$P(\tau) = \pi(a_0 \mid s_0)\pi(a_1 \mid s_1)\pi(a_2 \mid s_2)\cdots$$

Products of many small probabilities become extremely tiny. Logs turn products into sums:

$$\log P(\tau) = \sum_t \log \pi(a_t \mid s_t).$$

This is why machine learning code often stores:

$$\log \pi(a_t \mid s_t)$$

instead of directly storing:

$$\pi(a_t \mid s_t).$$

Logs are numerically stable and mathematically convenient.

6 Log Loss

Suppose we have a classification problem with the correct label y. The model assigns probability p_y to the correct class.

The log loss is:

$$\mathcal{L} = -\log p_y.$$

If the model assigns high probability to the correct answer, the loss is small:

$$-\log(0.99) \approx 0.01.$$

If the model assigns low probability to the correct answer, the loss is large:

$$-\log(0.01) \approx 4.61.$$

So log loss punishes confident wrong predictions strongly.

7 Cross Entropy

Cross entropy is the expected negative log-probability of the correct outcome. For a one-hot label, cross entropy is the same as log loss:

$$H(y, p) = -\log p_y.$$

In PyTorch:

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[0.2, 1.1]]) # batch of one example
target = torch.tensor([1]) # correct class is 1

loss = F.cross_entropy(logits, target)
print(loss)
```

The important connection to RL is:

policy gradient is like reward-weighted log loss on sampled actions.

8 Expectation

An expectation is a probability-weighted average. If X is a discrete random variable:

$$\mathbb{E}[X] = \sum_x P(X = x)x.$$

Example: if an action gives reward 1 with probability 0.7 and reward 0 with probability 0.3, then:

$$\mathbb{E}[R] = 0.7 \cdot 1 + 0.3 \cdot 0 = 0.7.$$

In RL, the objective is usually:

$$\boxed{J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]}$$

This says: choose policy parameters θ so that trajectories sampled from the policy have high expected reward.

9 Trajectory

A trajectory is one full sequence of interaction with the environment:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots).$$

For CartPole, one trajectory is one episode. It begins when the pole is reset and ends when the pole falls, the cart goes out of bounds, or the time limit is reached.

10 Return

The return is the total future reward. The simplest return is:

$$G_t = r_t + r_{t+1} + r_{t+2} + \cdots .$$

More commonly, we use discounted return:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

Here:

$$0 \leq \gamma \leq 1.$$

The parameter γ is the discount factor. It controls how much the agent cares about future reward.

```
def discounted_returns(rewards, gamma=0.99):
    returns = []
    total = 0.0

    for reward in reversed(rewards):
        total = reward + gamma * total
        returns.append(total)

    return list(reversed(returns))
```

11 Value Function

The value function estimates how good a state is. It is the expected return from that state:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Read this as:

$$\boxed{V^\pi(s) = \text{expected future reward if we start at state } s \text{ and follow policy } \pi.}$$

For CartPole, a state with the pole almost vertical should have a higher value than a state where the pole is about to fall.

12 Action-Value Function

The action-value function, or Q-function, estimates how good it is to take a specific action in a specific state:

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Read this as:

$$\boxed{Q^\pi(s,a) = \text{expected future reward after taking action } a \text{ in state } s.}$$

The difference between V and Q is:

$$V^\pi(s) = \text{value of a state}, \qquad Q^\pi(s,a) = \text{value of a state-action pair}.$$

13 Advantage

The advantage tells us whether an action was better or worse than the usual action from that state:

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

If:

$$A^\pi(s,a) > 0,$$

then action a was better than expected.
If:

$$A^\pi(s,a) < 0,$$

then action a was worse than expected.
This is extremely important in policy gradient. Instead of saying:

$$\text{increase probability of every action in high-return episodes,}$$

we can say:

$$\text{increase probability of actions that were better than expected.}$$

14 Entropy

Entropy measures how random a distribution is. For a discrete policy:

$$H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s).$$

If the policy is almost deterministic:

$$\pi(\cdot \mid s) = [0.99, 0.01],$$

entropy is low.
If the policy is uncertain:

$$\pi(\cdot \mid s) = [0.5, 0.5],$$

entropy is high.
In RL, entropy is often added as a bonus to encourage exploration:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} - \beta H(\pi).$$

```
dist = Categorical(logits=logits)
entropy_bonus = dist.entropy().mean()
loss = policy_loss - 0.01 * entropy_bonus
```

15 KL Divergence

KL divergence measures how different one probability distribution is from another:

$$D_{\text{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

In modern RL, especially RL for language models, KL divergence is used to prevent the new policy from moving too far from a reference policy.

KL penalty = do not let the updated policy drift too much.

16 Common Terms Summary

Term	Symbol	Meaning
State	s_t	Information observed at time t
Action	a_t	Choice made by the agent
Reward	r_t	Immediate feedback from environment
Policy	$\pi(a \mid s)$	Distribution over actions
Trajectory	τ	Sequence of states, actions, rewards
Return	G_t	Discounted future reward
Value	$V(s)$	Expected return from a state
Q-value	$Q(s, a)$	Expected return from a state-action pair
Advantage	$A(s, a)$	How much better an action was than expected
Entropy	$H(\pi)$	Randomness of a policy
KL divergence	D_{KL}	Difference between distributions

17 Minimal Mental Model

The shortest way to remember this lesson is:

> RL learns from random experience by assigning probabilities to actions and rewards to outcomes.

The policy gives probabilities. The environment gives rewards. The learning rule connects them through log-probabilities:

> good outcome $\Rightarrow$ increase log-probability of chosen actions.